ELEC-H310 – Digital electronics
Th02: C Language 2/3

Dragomir Milojevic
Université libre de Bruxelles

v3.3

Today

1. Pointers
2. Arrays
3. User defined data structures
4. Dynamic memory allocation
5. File access

1. Pointers

What is a pointer?

- Pointers are variables like any other: they store data; difference with normal variables is that a pointer store an address of the system memory (one pointer = one address of one byte location)
- We say pointers point to data: a pointer is always associated to some data type; Why do we need to a data type here?
- With pointers we distinguish: the address (var name) and the data stored at that address (var name preceded with $\star$ symbol)

```c
#include <stdio.h>

int main (void) {
   int  *ip; // pointer declaration to an int
   ...       // some code that initializes ip

   // Access address stored in pointer in hex format
   printf("Address stored in ip variable: %x\n", ip);
   // Access the value using the pointer
   printf("Value of *ip variable: %d\n", *ip);
}
```

Every variable has an address!

- Because every variable that is stored in the memory must have an address, we should be able to retrieve the address of any arbitrary variable (variable name is just a symbolic address)
- To access the address of any variable we use the operator `&` simply placed in front of the variable name, example:

```c
#include <stdio.h>

int main (void) {
   int  var1;

   printf("Address of var1 variable: %x\n", &var1);
   // var1 is the normal variable
   // but we can access the address of var1
   // so far we have been interested in var name only
   return 0;
}
```

- In the above how many hex digits do you expect to see & why?

Complete example of pointer usage

```c
#include <stdio.h>

int main (void) {

   int  var = 20;    // var declaration
   int  *ip;         // pointer declaration
                     // Attention !!!
                     // Non initialised pointer => it can point
                     // to any address, even something important

   ip = &var;        // store address of var in pointer

   printf("Address of var variable: %x\n", &var);

   /* address stored in pointer variable */
   printf("Address stored in ip variable: %x\n", ip);

   /* access the value using the pointer */
   printf("Value of *ip variable: %d\n", *ip);

   return 0;
}
```

Address of a pointer

- Since pointers are variables stored in the computer memory, we could also apply the operator `&` to get the address of a pointer variable itself

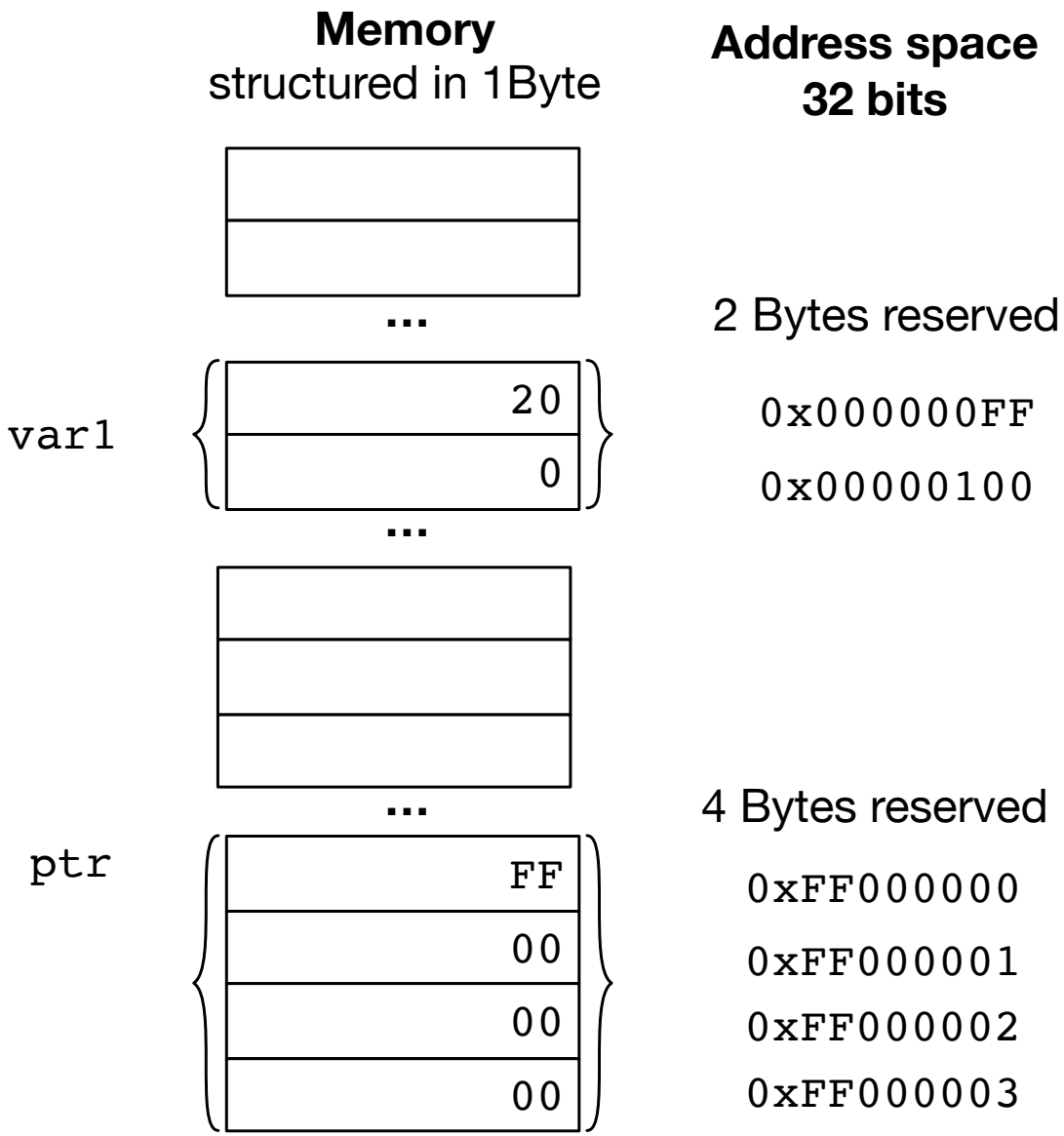
```
1  #include <stdio.h>
2
3  void main (void) {
4     int  a=20;                          // some var
5     int  *ip1;                          // pointer1
6     int  *ip2;                          // pointer2
7
8     ip1 = &a ;                          // store some value
9     ip2 = &ip1 ;                        // store ptr address
10    printf("1:%x\n2:%x\n", &ip1, &ip2 );  // What do you see here?
11    printf("1:%d\n2:%x\n", *ip1, *ip2 );
12    // What do you see here? Why *ip2 is hex?
13
14    ip2 = ip1;
15    printf("1:%d\n2:%d\n", *ip1, *ip2 );  // What do you see here?
16 }
```

- In the example above pointer `ip2` first contains the address of the pointer `ip1`; `ip1` &`ip2` are different in line 10
- But the printed value on line 15 is the same due to line 14!!!

Pointers and memory – illustration

```
short int var1=20

     &var1=0x000000FF
```

```
short int *ptr
ptr=&var1

     ptr= 0x000000FF

     *ptr= 20

     &ptr=0xFF000000
```



One specific pointer: NULL

- One of the header files (typically `stdio.h`) defines a very specific pointer value called `NULL`
- This value is guaranteed not to be a valid address in memory

```
1  #include <stdio.h>
2
3  int main () {
4     int  a=20;
5     int  *ptr = NULL; // pointer is declared and
6                       // assigned default value
7     ptr = &a;
8     if (ptr == NULL) exit(0);
9
10    printf("The value of ptr is : %x\n", ptr);
11 }
```

- Pointers are typically assigned to `NULL` during their declaration; afterwards we compare the current pointer value to `NULL` to see if the pointer contains a valid address yes or no
  - ▹ In the example above `ptr` is assigned at line 7 & tested at line 8

Why it is important to use valid memory address?

- In the previous example, the line:

```
8    if (ptr == NULL) exit(0);
```

  was not needed because we had in the line before:

```
7    ptr = &a;
```

- In another words we had a valid address in the pointer `ptr`; but this may not be the case (anytime before the assignment in line 7)
- What will happen if we use a non-valid address to access data will depend on the type of operation
  - ▹ If we read data from a non-valid address, this could work
  - ▹ If we write data to a non-valid address, it could be a BIG problem!; you could write anywhere & erase something sensitive![1]

Advice! Always assign NULL and check!

[1] Modern OSs will protect memory to a certain extent

Pointers, functions calls and arguments 1/2

Look at the code below:

```
1  void ChangeI(int i);
2  main() {
3     int i=0;                    // some local variable
4     ChangeI(i);                 // that we pass to a function
5     printf("i: %d\n", i);       // what do you expect to see?
6  }
7  void ChangeI(int i) {
8     i=10;                       // Function modifies the argument
9                                 // is variable modified when returning?
10 }
```

- Despite the common sense saying that the above program is ok: this will never work in practice!
- This is because the variable `i` in `ChangeI`, when passed as an argument, is a copy of `i`, not the variable itself; in this case we say that the argument is passed by value
- The copy of `i` used in `ChangeI` will be destroyed on exit from function, so in function `main`, the variable `i` will remain unchanged; in terminal we will see 0 after executing line 5

Pointers, functions calls and arguments 2/2

And now:

```
1 void ChangeI(int *i);
2 main() {
3   int i=0;
4   ChangeI(&i);              // we now pass the address to the function
5 }
6 void ChangeI(int *i) {
7   *i=10; // we access the value of the address
8 }
```

- This one will work! – we pass the pointer, i.e. the address of the variable; we sat that the argument is passed by reference
- Since this is a reference, i.e. the absolute memory location, it can be accessed anywhere in the program & we can do whatever we want with the content of it, including modifying it persistently
- Another advantage: we can pass an arbitrary number of arguments to a function even if we don't know the number of variables to pass at compile-time; Can you explain?

Pointers, addresses and local variables

- Consider the code below. It does make sense at a first glance; however it does hide a serious flaw; Can you say what?

```
#include<stdio.h>

void DanglingPointer(int *x);

int main(void) {
   int *p;
   DanglingPointer(p);
   return 0;
}

void DanglingPointer(int *x) {
   int y=0;
   x = &y;
   return;
}
```

2. Arrays

One dimensional arrays

- In C you can define an array of variables of the **same data type**, that will be stored in memory as a contiguous list of elements
- Size of the area must be supplied to determine the amount of memory that needs to be reserved when variable is declared:

```
1 int myArray[100], myArray2[100];
2 myArray[0]  = 17;            // set first element
3 myArray[99] = 47;            // set last element
4 myArray2[0] = 15;            // set first element
```

  ▹ Here `myArray` needs `sizeof(int)` $\times$ 100 bytes
- First element in the array is indexed with a 0, so in the above index 99 corresponds to the 100th (last) element in the array
- Write access to an element outside of the array range could cause problems since the program could erase some vital information
- What will happen in the above example if we add as line 5?

```
1 myArray[100]=59;
```

Manipulating 1D array

- Following program will do some simple arithmetic using 3 arrays:

```
int main(void) // Main
{
   int A[100],B[100],C[100];

  // option 1
      for(i=0;i<100;i++)
         C[i]=A[i]+B[i];
  // option 2
      for(i=1;i<=100;i++)
         C[i-1]=A[i-1]+B[i-1];
   return 0;
}
```

- Make sure you can:
  - Explain the above `for` loops in details
  - Discuss index in both cases
  - What do we miss here?

Multi dimensional arrays

- Multi-dimensional arrays could be defined as follows (2D here):

```
int myArray[10][10];
myArray[0][0] = 17;
myArray[5][5] = 47;
```

- Memory space is contiguous, so in realty multi-dimensional arrays are one 1D block of data in the memory, so previous variable declaration is equivalent to:

```
int myArray[100];
board[0] = 17;
board[5+5*10-1] = 47;   // note how do we access element 5,5 in 1D array
```

- In general any element `(i,j)` of the array is accessed using:

$$\texttt{i} + \texttt{j} \times \texttt{sizeof(arrayX)} - 1$$

where `i,j` are respectively row and column indexes of 2D array and `arrayX` (starting at 0) is the "width" of the 2D array

Array simple example: get, store & print an array

```c
#include <stdio.h>

int main() {

  int values[5];

  printf("Enter 5 integers: ");

  // taking input and storing it in an array
  for(int i = 0; i < 5; i++) {
     scanf("%d", &values[i]);
  }

  printf("Displaying integers: ");

  // printing elements of the array
  for(int i = 0; i < 5; i++) {
     printf("%d\n", values[i]);
  }
  return 0;
}
```

Array simple example: average of an array

```c
#include <stdio.h>

int main() {
  int marks[10], i, n, sum = 0;
  double average;

  printf("Enter number of elements: ");
  scanf("%d", &n);

  for(i=0; i < n; i++) {
    printf("Enter number%d: ",i+1);
    scanf("%d", &marks[i]);
    // adding integers entered by the user to the sum variable
    sum += marks[i];
  }

  // explicitly convert the sum to double
  // then calculate average
  average = (double) sum / n;

  printf("Average = %.2lf", average);
  return 0;
}
```

Array with pointers

- Arrays can be accessed using pointers:

```c
#include <stdio.h>

int main() {
   int myNumbers[5] = {25, 50, 75, 100, 33};
   int *ptr = myNumbers;
   int i;
   for (i = 0; i < 4; i++) {
     printf("%d\n", *(ptr + i));
   }
}
```

- In the above example `ptr` holds the address of the 1st element of `myNumbers`; similar things can be achieved in different ways:

```c
   int myNumbers[5] = {25, 50, 75, 100, 33};
   int *ptr = myNumbers;
   int *ptr = &myNumbers[0];       // This is the same as above
   int i=3;
   printf("%d\n", myNumbers[i]);
   printf("%d\n", *(ptr + i));     // This is the same as above
```

- Pointers are heavily used in arrays

2D-Array matrix multiplication: the function

- A function is implemented in say MatMul.c

```c
#include <stdio.h>
#include <stdlib.h>

// matrix dimensions so that we dont have to pass them as
// parametersmat1[R1][C1] and mat2[R2][C2]
#define R1 2 // number of rows in Matrix-1
#define C1 2 // number of columns in Matrix-1
#define R2 2 // number of rows in Matrix-2
#define C2 3 // number of columns in Matrix-2

void multiplyMatrix(int m1[][C1], int m2[][C2])
{
    int result[R1][C2];
    printf("Resultant Matrix is:\n");

    for (int i = 0; i < R1; i++) {
        for (int j = 0; j < C2; j++) {
            result[i][j] = 0;
            for (int k = 0; k < R2; k++) {
                result[i][j] += m1[i][k] * m2[k][j];
            }
            printf("%d\t", result[i][j]);
        }
    }
}
```

2D-Array matrix multiplication: main function

- The `main` function calling the function of the previous slide; saved in MyProg.c
  - ▷ Something is missing! Code will not compile. Why? Solve ...

```c
#include <stdio.h>
#include <stdlib.h>

void main(void) {
    // R1 = 4, C1 = 4 and R2 = 4, C2 = 4; update these values in MACROs
    int m1[R1][C1] = { { 1, 1 }, { 2, 2 } };
    int m2[R2][C2] = { { 1, 1, 1 }, { 2, 2, 2 } };

    // if column of m1 not equal to rows of m2
    if (C1 != R2) {
        printf("The number of columns in Matrix-1 must be "
               "equal to the number of rows in "
               "Matrix-2\n");
        printf("Please update MACROs value according to "
               "your array dimension in "
               "#define section\n");
        exit(EXIT_FAILURE);
    }
    // Function call
    multiplyMatrix(m1, m2);
}
```

3. User defined data structures

User defined (composite) data types

- Users can create **composite variables** i.e., group of data stored in one place – structures (structs in short)
- Structures can contain a fixed number of elements (decided at compile time) but of different data types (contrary to arrays)
- Different data elements within the structure are referred to as structure members
- Like any other variables, structures are identified using variable names; they can be also accessed through pointers
- Structures are used to simplify data manipulation, improve code readability and code re-use
- Usage – we first need to define a data structure and then define a variable of that type; different members are then accessed:
  - Using (dot) `.` operator if the structure is defined as normal variable (e.g. `mystry.name`)
  - If declared as a pointer Using (arrow) `->` operator if the structure is defined as pointer (e.g. `mystry->name`)

Structures – usage with normal variables

- We first need to define a data structure:

```
struct structure_name {
    data_type1 member1;
    data_type2 member2;
    data_type2 member3;
    ...
};
```

- We then need to define a variable of that type

```
structure_name var_name;
```

  ▷ Attention: variable will now be allocated in the stack, so that we can effectively access the data!

- Different members can be then accessed using (dot) . operator:

```
var_name.member1=5;
var_name.member2="MyData";
var_name.member3=3.14;
```

Structures – usage with with pointers

- Same structure as previously:

```
struct structure_name {
    data_type1 member1;
    data_type2 member2;
    data_type2 member3;
    ...
};
```

- But instead of the normal variable we use pointers:

```
structure_name *var_name;
```

  - Attention: variable will NOT be allocated, so with the above we can't effectively access the data!

- Different members can be accessed using (arrow) -> operator:

```
var_name -> member1;
var_name -> member2;
var_name -> member3;
```

Structures & pointers

- If we would like to use the pointers we should do something like:

```
1  struct structure_name {
2      data_type1 member1;
3      data_type2 member2;
4      data_type2 member3;
5  };
6
7  structure_name var_name;
8  structure_name *var_ptr;
9
10 var_ptr = &var_name;
11 printf("%d", sizeof(var_name));
```

- Now we do have the memory space allocated for one variable of type `structure_name`, this is `var_name`
- The pointer `var_ptr` points to an address in the stack, where a sequence of bytes has been reserved for `var_name`; the number of the bytes could be of course computed with the function `sizeof` (line 11)
  - ▹ Next chapter will show how we can really use structures with pointers

Simple example

```c
struct tag {                          // Struct definition
   char last_name[20];
   char first_name[20];
   int age;
   float rate;
};

struct tag my_tag;                    // Global variable defined

int main(void){
   strcpy(my_struct.lname,"Jensen");  // Access in write mode
   strcpy(my_struct.fname,"Ted");
   printf("\n%s",my_struct.fname);
   printf("%s\n",my_struct.lname);    // Access in read mode
   return 0;
}
```

Example of structures with pointers

```c
#include <stdio.h>
#include <string.h>
struct tag{
   char lname[20];
   char fname[20];
   int age;
   float rate;
};
struct tag my_struct;
void show_name(struct tag *p);

int main(void) {
   struct tag *st_ptr;
   st_ptr = &my_struct;
   strcpy(my_struct.lname,"Jean");      // Fill in some data
   strcpy(my_struct.fname,"Tean");
   printf("\n%s ",my_struct.fname);
   printf("%s\n",my_struct.lname);
   my_struct.age = 63;
   show_name(st_ptr);                   // Pass the pointer
   return 0;
}
void show_name(struct tag *p) {
   printf("\n%s ", p->fname);
   printf("%s ", p->lname);
   printf("%d\n", p->age);
}
```

4. Dynamic memory allocation

More on memory

- We know that memory, seen by the executed program, is a contiguous 1D space with addresses ranging from, say for the sake of the example: `0x00000000` to `0xFFFFFFFF` in increments of 1
  - How many bits the address above will need?
  - What is the size of the memory above assuming 1 byte per address?
  - How actual physical memory maps to this space?
- Each memory location stores 1 byte, this is a HW choice, it could be something else; 1B per address is the most common
  - You should be able by now to explain why.
- Everything related to the current execution context of a program is saved in the memory: the program itself, all the (raw) data & the execution context of each function
- Memory will store other things too: data related to OS, & eventual other programs that run (case of multi-threaded OSs)
- That is many things, so we need an efficient way to manage this

Memory organisation

- Memory is split into distinct parts: bottom part is reserved for the code itself and top part, i.e., user data is split into two parts:
  - Stack – we know this, all local vars go there
  - Heap – possibly large memory space, user managed

Growth ↓	Stack	Auto managed
	Free space	
Growth ↑	Free space	Dynamic memory
	Heap	
Fixed at compile time	Uninitialized data segment	Global & static data
	Initialised data segment	
	Code segment	Read only

- Note the opposite directions of growth for stack and heap
- Above representation is not to scale; heap could be much bigger than the stack (almost all addressable memory space)

Stack – revisited

- Stack is a memory region that stores temporary variables created by each function, including `main()`
- Stack is managed by the CPU in LIFO fashion (Last In, First Out)
  - When a function declares new variable, it is pushed onto the stack
  - When we exit a function, all variables pushed onto stack by that function are freed, i.e. **deleted** (stack variables are local)
  - Once a stack variable is freed, that region of memory becomes available for other stack variables
- Memory management is automated, the programmer does not have to allocate or free the stack memory explicitly
- CPU organises stack efficiently & RD/WR ops. to stack are fast
- This approach is extremely handy since it enables memory management automation well suited for computations on smaller data sets with lots of functions

Static variables

- Not all variables are erased when the function returns → static variables; they preserve their value even if out of scope!
- In another words a static variable will preserve previous value from the previous scope; i.e., it will not be initialised again in the new scope (e.g. after two successive function calls); example:

```c
#include<stdio.h>

int sf(void) {
  static int count = 0;
  count++;
  return count;
}

int main(void) {
  printf("%d, ", sf());
  printf("%d, ", sf());
  printf("%d  ", sf());
  return 0;
}
```

- Execution of the program above will produce: `1, 2, 3`

Register variables

- Access to internal CPU (core) registers is much faster than the access to central or cache memories
- To force the compiler to store data into one of the available registers, one can use register variables; compiler may, or may not use this
- Expected outcome is that frequently used variables, if stored locally, will improve program performance since less time will be spent to fetch the data from a remote memory location
- Attention: you can not use the address of the variable declared as registered one; registers do not belong to normal memory space

```
1  #include <stdio.h>
2
3  int main() {
4      // Declare a reg variable and init to 10
5      register int i = 10;
6      // Assign address of var i to pointer a
7      int *a = &i;
8      printf("%d", *a);
9      return 0;
10 }
```

- In the above code line 11 will produce an error

Practical stack considerations

- Stack size is fixed for a given program during compile-time and can't be changed during program execution; in another words if you want the program to use bigger stacks it needs to be recompiled using another compiler setting
- Default stack size is typically few to few tens of kBytes of memory only, not more (check your compiler default stack size)
- Small stack size allows certain depth of successive – nested function calls: function that call other function that call other function, and so on
  - Assume memory of 16GB and stack of 1MB, how many nested functions calls you will be able to perform?
- Also true for recursive functions: functions that call them selfs, we will see some examples of these in the next lecture
- What will happen if we try to allocate more then what is made available by the stack definition?

Stack limitations

- Stack have fixed size, same for all functions, no matter how much data function really needs
- Important consequences of this illustrated using arrays in C:
  - We need to know the array size at compile-time: even if data type is given, what do we do when we don't know how many elements to allocate upfront:
    - Arrays could have size that is computed during execution
    - A solution could be to allocate maximum array size (something fairly big), no matter how much elements we will really need; but this would be fairly inefficient
  - How to deal with huge data sets?
    - We could think of increasing the stack size, but then we will be waisting a lot of memory resources in functions that do not require so much memory
- Solution: use heap through dynamic memory allocation, explicit memory management done in co-operation with OS & HW

Heap memory & allocation

- To handle data sets bigger than typical stack size, or if the size of the data set is not known during the compile time we use heap
- Heap could be the whole addressable memory space
- Size of addressable memory space, fixed at computer design time depends on the number of bits of addresses that we chose; this is fixed by HW, at CPU/computer design time
  - Example: assume 32-bits of address, this is 2^{32}, so 4294967296Bytes, or 4194304kB, or 4096MB or 4TB
- Heap is accessed through dynamic memory allocation, meaning that we do that at run-time, during program execution, using standardised high-level C functions
- For allocation we have a choice between:
  - `malloc()` – memory is not initialised, so you could have traces of previous execution, or
  - `calloc()` – initialises all memory locations to 0; potentially slower to execute than `malloc()`

Heap memory & management

- **Used memory should and must be freed after use:**
  - ▹ `free()` – deallocates memory not needed any more
- It is mandatory to free some memory at one point in time to allow future function calls to be able to allocate some; computers do have finite amount of memory compared to Turing machines
- Forgetting to free memory will result in memory leaks
- Depending on the amount of memory leaks, and the time your program run, computer will sooner or later run out of memory
- This will cause program to crash, or to exit gracefully, depending on how well you write your code
- Note the memory leaks will not show themselves systematically, they are difficult to debug, and it is in general a very good idea to avoid them at any cost!
- You will see here how to make sure that your program exits gracefully, even if there is no more free memory

Actual memory management & virtual memory

- So, it is the job of the programmer to plan memory usage and make sure that things run smoothly
- In reality the SW side of memory allocation is only about communicating with OS; the OS will on his turn talk to HW to make sure that the memory is allocated/freed during execution
- Low-level memory management is still done by HW, not even OS
- This concept can be pushed even further with the concept of virtual memory: actual physical (DRAM) memory space is mapped to a virtual memory space that is seen by HW/OS and our application
  - ▹ This is why you could allocate potentially the whole addressable memory space even in machine with limited amount of memory.
- The executable sees the complete memory space (all addressable locations); and we let HW to offload parts of the memory content to our HDD; this is why an SSD HDD can help to reduce the execution time of a memory hungry application (they can reduce swap time: memory to HDD transfers)

Practical dynamic memory allocation

- Performed using the `malloc()` function
- Header file: `stdlib.h`
- Syntax: `void *malloc(size_t size)`
- Parameter `size_t size` – the number of bytes to allocate
  - ▹ `size_t` is an `unsigned int` type, guaranteed to be big enough to contain the size of the biggest object the host system can handle; it is system/compiler dependent
  - ▹ If the compiler is 32 bits, then it is an `unsigned int`; `size_t` is a simple `typedef` (alias) to `unsigned int`
  - ▹ If the compiler is 64 bits, then `size_t` is a `typedef` for `unsigned long long`
- Return value
  - ▹ If the allocation succeeds, the functions returns a pointer **to the beginning** of the newly allocated memory block (first byte)
  - ▹ If the allocation fails, then it returns a `NULL` pointer
  - ▹ It is essential that we use NULL to test if malloc went well

Dynamic memory reallocation

- If we need to dynamically re-adjust already allocated memory block, we can use `realloc` function
  - ▹ Syntax: `void *realloc(void *ptr, size_t size)`
- In the above `ptr` is a previously allocated pointer
- Realloc is used to optimise memory usage; e.g. if you have many large arrays that at run-time can be squeezed at some point in time
  - ▹ Discuss what will happen if you do not re-adjust the array sizes?
- Works only on dynamically allocated pointers, below will not work

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main() {
4     int arr[10];          // initialize an array of size 10
5     int *ptr = arr;       // store the array to pointer
6     int *new_ptr;         // new pointer for reallocating
7     arr[0] = 5;
8     arr[1] = 10;
9     new_ptr = (int *)realloc(ptr, sizeof(int)*5);    // use the new pointer
10    *(new_ptr + 2) = 30;
11    for(int i = 0; i < 5; i++){
12       printf("%d ", *(new_ptr + i));
13    }
14 }
```

Example of dynamic memory allocation: arrays

```c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main (void) {
5     int i,n;
6     char *buffer;            // pointer that will point to sting array
7                              // the length of a string is user defined
8     printf ("How long do you want the string? ");
9     scanf ("%d", &i);
10                             // malloc returns a void pointer we need to cast
11    buffer = (char *) malloc ((i+1)*sizef(char));
12                             // graceful exit!
13                             // in case we have memory leaks elsewhere
14    if (buffer==NULL) {printf ("No memory!"); exit (1);}
15
16                             // fill array with random strings
17    for (n=0; n<i; n++)
18      buffer[n]=rand()%26+'a';
19                             // append end character
20    buffer[i]='\0';
21    printf ("Random string: %s\n",buffer);
22                             // we free unused memory in the end, i+1 in line 11
23    free (buffer);
24    return 0;
25 }
```

Example of dynamic memory reallocation

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
   // Initially allocate memory for 2 integers
   int *ptr = malloc(2 * sizeof(int));
   if (ptr == NULL) {
      fprintf(stderr, "Memory allocation failed\n");
      return 1;
   }
   ptr[0] = 5;
   ptr[1] = 10;
   // increase the size of the integer
   int *ptr_new = realloc(ptr, 3 * sizeof(int));
   if (ptr_new == NULL) {
      fprintf(stderr, "Memory reallocation failed\n");
      free(ptr);
      return 1;
   }
   ptr_new[2] = 15;
   int i;
   for (i = 0; i < 3; i++) {         // Display the array
      printf("%d ", ptr_new[i]);
   }
   free(ptr_new);
   return 0;
}
```

Example of dynamic memory allocation: structs

```c
struct list {
  int *data;    // Points to the memory where the list items are stored
  int numItems; // Indicates how many items are currently in the list
  int size;     // Indicates how many items fit in the allocated memory
};
// Definition of another function
void addToList(struct list *myList, int item);

int main() {
  struct list myList;
  int amount;
  // Create a list and start with enough space for 10 items
  myList.numItems = 0;
  myList.size = 10;
  myList.data = malloc(myList.size * sizeof(int));
  // Find out if memory allocation was successful
  if (myList.data == NULL) {
    printf("Memory allocation failed");
    return 1; // Exit the program with an error code
  }

  // Add any number of items to the list specified by the amount variable
  amount = 44;
  for (int i = 0; i < amount; i++) {
    addToList(&myList, i + 1);
  }
// Continues on the next page
```

Example of dynamic memory allocation: structs

```
29    // Continued from previous page
30    // Display the contents of the list
31    for (int j = 0; j < myList.numItems; j++) {
32      printf("%d ", myList.data[j]);
33    }
34
35    // Free the memory when it is no longer needed
36    free(myList.data);
37    myList.data = NULL;
38
39    return 0;
40  } // Here ends main function
41
42  // This function adds an item to a list
43  void addToList(struct list *myList, int item) {
44    // If the list is full then resize the memory to fit 10 more items
45    if (myList->numItems == myList->size) {
46      myList->size += 10;
47      myList->data = realloc( myList->data, myList->size * sizeof(int) );
48    }
49
50    // Add the item to the end of the list
51    myList->data[myList->numItems] = item;
52    myList->numItems++;
53  }
```

5. File access

File access

- We want to be able to access the file system of the host machine
- This is managed by the OS that interacts with the appropriate HW, in this case the HDD controller
- C embeds high-level functions to access files; these functions are standardised (ANSI-C), their implementation is architecture / OS specific (& we do not care about these ...)
- Access to files is performed in three phases:
  - We first open the file, with certain access type: read, write or append; write erases the file, if it exists; append just adds the data on top of what the file already contains
    - Access types are crucial since OS should lock the file upon opening for write to avoid race conditions: two programs should not open the same file for write at the same time; **write access to files must be exclusive**
  - Once the file is open, we can read/write data to it
  - Once completed, we should "close" the file; this will release the file status in the eyes of the OS; if the file was open for write, it will become again accessible for others to access it

Open file for reading & read data

- Using `fopen()`, defined in `stdio.h` header file
- Arguments: file name and the access type (`r`, `w`, `a`, etc.); returns pointer to the (data) stream, or `NULL` pointer if file not opened
- We use `fread()` function to **copy** `size` elements from the opened file to `source` pointer, so somewhere in memory

```
//Open some files and define a pointer to the file
FILE *fp = fopen("data.test","r");
// check if file has been successfully opened, always do this
if (fp!=NULL) {
   printf("File opened ... \n");
   fread(source, sizeof(unsigned char), size, fp);
   printf("File read! \n");
   fclose(fp);
} else {
   printf("Can't open specified file!\n"); // file not found, exit
   exit(0);
}
```

What is missing & what happens during execution?

Open file for writing

- Change the argument for `fopen`, access type is now `w`
- We also need to provide information on what to copy:
  - ▹ from where in memory, `destination` pointer
  - ▹ and how many elements `size`
- Once we are finished we need to close file pointer(s), and this should be done for both read & write file operations

```
FILE *fp2 = fopen("res_normal.raw","w");     // pointer to file
if (fp2!=NULL) {                             // test
   printf("File opened for write ... \n");
   fwrite(destination, sizeof(unsigned char), size, fp2);
   printf("File written! \n");
   fclose(fp2);
} else {
   printf("Can't open specified file!\n");
   exit(0);
}
```

Most of the tips in the above code: you should be able to figure them out by **reading documentation** on C-functions

Learn how to read documentation of C-functions

```
size_t fread(
   void *buffer,
   size_t size,
   size_t count,
   FILE *stream
);
```

Parameters

`buffer`
Storage location for data.

`size`
Item size in bytes.

`count`
Maximum number of items to be read.

`stream`
Pointer to `FILE` structure.

Return Value

`fread` returns the number of full items actually read, which may be less than `count` if an error occurs or if the end of the file is encountered before reaching `count`. Use the `feof` or `ferror` function to distinguish a read error from an end-of-file condition. If `size` or `count` is 0, `fread` returns 0 and the buffer contents are unchanged. If `stream` or `buffer` is a null pointer, `fread` invokes the invalid parameter handler, as described in Parameter Validation. If execution is allowed to continue, this function sets `errno` to `EINVAL` and returns 0.

See _doserrno, errno, _sys_errlist, and _sys_nerr for more information on these, and other, error codes.